

claude-wsl-ubuntu / 4f364eb7-a600-4a2c-9b37-e5a313b6db9d

Metadata

```
- Source: `claude-wsl-ubuntu`  
- Kind: `claude`  
- Source file: `\\wsl.localhost\Ubuntu\home\avidullu\.claude\projects\home-avidullu-projects-khelsutra-guru\4f364eb7-a600-4a2c-9b37-e5a313b6db9d\subagents\workflows\wf_b0aa0abe-638\agent-a7d7084767b9679cf.jsonl`  
- SHA-256: `ce8d079aa4a83772a5237833aed6dc39de27fe40c16ccc188f21501ce5b8d9ba`  
- Source modified: `2026-06-29T17:04:19+00:00`  
- Imported at: `2026-07-05T16:48:30+00:00`  
- project: `wf_b0aa0abe-638`  
- session_id: `4f364eb7-a600-4a2c-9b37-e5a313b6db9d`
```

Transcript

1. user (2026-06-29T17:01:41.449Z)

In /home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer, produce the fix plan for issue #398: the served Gate-A regression litmus crashes with `AttributeError: 'dict' object has no attribute 'fusion'` at backend/pipeline/segmenters/fusion_hybrid.py:135 when run on real golden data.

Read: backend/eval/served_gate_a.py (esp _served_idx_cfg + served_handoff_windows), backend/pipeline/segmenters/fusion_hybrid.py::_enrich_candidate_stats (how it reads idx_cfg.fusion / fcfg.cue_flags), how production process_video builds the idx_cfg it passes (find the typed AppConfig/IndexingConfig path ? confirm production passes a TYPED config), and tests/test_served_gate_a_regression.py (note every pinned BAND/number, e.g. mean_delta_f1 range, the 2026-06-22 magnitudes).

Decide the cleanest fix: (A) make _served_idx_cfg build a typed IndexingConfig (preferred ? exercises the real contract), vs (B) make _enrich_candidate_stats tolerate dict-or-typed. Give the EXACT code change. State whether the pinned test bands will likely change when the litmus actually runs on the 15-clip golden set (so they may need re-pinning), and exactly what re-measuring needs (SERVED_GATE_A_MANIFEST env ? /home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer/output/p2b_golden/manifest_local.json which already exists; cv2 importable). Note coverage impact (these tests currently skip; fixing makes them runnable on the owner box, but CI still skips without golden ? so don't make CI depend on golden).

2. assistant (2026-06-29T17:01:43.719Z)

I'll analyze the issue by reading the relevant files. Let me start by exploring the codebase structure and reading the key files.

3. user (2026-06-29T17:01:45.164Z)

```
1      """SERVED Gate-A verification: run the *production* ``FusionSegmenter`` decision seams  
2      on a cached WASB trajectory and score the resulting AI-handoff windows vs golden labels.  
3  
4      The offline litmus (``backend.eval.ablation`` / ``fusion_compare``) measures Gate A on  
the  
5      **permissive proposal windowing** (``distill_local.PROPOSAL`` = recall-oriented) re-  
scored  
6      through the experiment harness's ``_gate_mask``. That validates the *signal*. It does  
NOT  
7      exercise the **served path**: in production the windows come from  
8      ``trajectory_to_action_windows`` at the production operating point  
9      (``velocity_threshold=0.02, dead_time_merge_gap=2.0, min_action_window_duration=2.0``),  
the  
10     net-crossing count is computed by ``FusionSegmenter._enrich_candidate_stats``, and the  
gate is  
11     ``FusionSegmenter._select_for_handoff``. THIS module drives exactly those production  
methods ?  
12     no reimplementatation of the windowing or the gate ? on a cached trajectory CSV, so the
```

```

13     verification reflects what ``process_video`` actually hands to Gemini.
14
15     GPU-free and Gemini-free: it stops at the handoff-selection boundary (the windows that
WOULD
16     be sent to the AI). With the person cue OFF (the default) nothing here loads torch or
touches
17     the GPU ? it just re-scores cached WASB trajectories.
18
19     Run (owner box, golden trajectories present):
20     python -m backend.eval.served_gate_a --manifest output/human_lovo_manifest.json
21     """
22
23     from __future__ import annotations
24
25     import argparse
26     import json
27     from dataclasses import dataclass
28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.eval import segmentation_metrics as _segm
31     from backend.eval.calibrate_wasb import load_golden_gts
32     from backend.eval.metrics import Interval, Score, score
33     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
34     from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
35
36     # Production windowing operating point (config.json indexing defaults ? the values
37     # TrajectoryHybridSegmenter.process_video reads for the served fusion run).
38     PROD_VELOCITY_THRESH = 0.02
39     PROD_MERGE_GAP = 2.0
40     PROD_MIN_WINDOW = 2.0
41
42
43     def _served_idx_cfg(min_crossings: int) -> Dict[str, Any]:
44         """The ``indexing`` config dict the served FusionSegmenter seams read ? mirrors
45         config.json's ``indexing.fusion`` block, with Gate A at ``min_crossings`` and the
46         person cue OFF (so no GPU / torch is touched)."""
47         return {
48             "fusion": {
49                 "substrate": "wasb",
50                 "min_crossings": min_crossings,
51                 "min_players": 0,
52                 "cue_flags": {},
53             },
54             "wasb": {"velocity_threshold": PROD_VELOCITY_THRESH},
55             "dead_time_merge_gap": PROD_MERGE_GAP,
56             "min_action_window_duration": PROD_MIN_WINDOW,
57         }
58
59
60     def served_handoff_windows(
61         trajectory_csv: str, fps: float, frame_width: float, min_crossings: int
62     ) -> List[Interval]:
63         """Run the PRODUCTION FusionSegmenter decision path on a cached trajectory and
return the
64         windows that would reach the AI handoff (i.e. the served rally candidates).
65
66         Uses the real ``FusionSegmenter`` seam methods ? ``_enrich_candidate_stats``
(computes
67         ``net_crossings``) and ``_select_for_handoff`` (Gate A) ? over windows from the
production
68         ``trajectory_to_action_windows`` operating point. No GPU, no Gemini: the person cue
is OFF.
69         """
70         points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
71         windows = TrackNetRunner.trajectory_to_action_windows(

```

```

72         points,
73         fps=fps,
74         frame_width=frame_width,
75         velocity_thresh=PROD_VELOCITY_THRESH,
76         merge_gap=PROD_MERGE_GAP,
77         min_window_duration=PROD_MIN_WINDOW,
78     )
79     idx_cfg = _served_idx_cfg(min_crossings)
80     # Construct the real served segmenter (db=None, config carries the indexing block).
We only
81     # call the pure decision seams ? no process_video, no DB, no provider ? exactly as
the
82     # registry constructs it for `name`/`description`.
83     seg = FusionSegmenter(None, {"indexing": idx_cfg, "sport": "badminton"})
84     # The served per-window stats (incl. net_crossings) via the production enrichment
hook.
85     idx_cfg_typed: Any = idx_cfg # type: ignore[annotation-type-mismatch]
86     window_stats = [
87         seg._enrich_candidate_stats(
88             cw, points, fps, frame_width, video_path="", idx_cfg=idx_cfg_typed
89         )
90         for cw in windows
91     ]
92     keep = seg._select_for_handoff(windows, window_stats, idx_cfg_typed)
93     return [(windows[i]["start"], windows[i]["end"]) for i in keep]
94
95
96 @dataclass
97 class ServedVideoResult:
98     name: str
99     num_gt: int
100     on: Score # Gate-A ON (min_crossings=2)
101     off: Score # Gate-A OFF (min_crossings=0, control)
102     seg_ratio_on: float # |preds| / |gts| with Gate-A ON (over-segmentation)
103     seg_ratio_off: float
104     n_handoff_on: int
105     n_handoff_off: int
106
107     @property
108     def delta_f1(self) -> float:
109         return self.on.f1 - self.off.f1
110
111
112 def evaluate_video(
113     spec: Dict[str, Any], iou: float = 0.5, on_crossings: int = 2
114 ) -> ServedVideoResult:
115     """Score the served Gate-A ON (min_crossings=on_crossings) vs OFF (0) handoff
windows for
116     one golden video against its golden labels."""
117     fps, fw = float(spec["fps"]), float(spec["frame_width"])
118     traj, labels = str(spec["trajectory"]), str(spec["labels"])
119     gts = load_golden_gts(labels)
120     preds_on = served_handoff_windows(traj, fps, fw, min_crossings=on_crossings)
121     preds_off = served_handoff_windows(traj, fps, fw, min_crossings=0)
122     return ServedVideoResult(
123         name=str(spec["name"]),
124         num_gt=len(gts),
125         on=score(preds_on, gts, iou),
126         off=score(preds_off, gts, iou),
127         seg_ratio_on=_segm.segment_count_ratio(preds_on, gts),
128         seg_ratio_off=_segm.segment_count_ratio(preds_off, gts),
129         n_handoff_on=len(preds_on),
130         n_handoff_off=len(preds_off),
131     )
132

```

```

133
134 def evaluate_manifest(
135     specs: List[Dict[str, Any]], iou: float = 0.5, on_crossings: int = 2
136 ) -> List[ServedVideoResult]:
137     return [evaluate_video(s, iou, on_crossings) for s in specs]
138
139
140 def aggregate(results: List[ServedVideoResult]) -> Dict[str, Any]:
141     """Mean ON/OFF F1 + over-seg ratio, the paired ?F1, and the per-video win count."""
142     n = len(results)
143
144     def _mean(getter) -> float:
145         return sum(getter(r) for r in results) / n if n else 0.0
146
147     deltas = [r.delta_f1 for r in results]
148     wins = sum(1 for d in deltas if d > 1e-9)
149     losses = sum(1 for d in deltas if d < -1e-9)
150     return {
151         "num_videos": n,
152         "mean_f1_on": _mean(lambda r: r.on.f1),
153         "mean_f1_off": _mean(lambda r: r.off.f1),
154         "mean_delta_f1": _mean(lambda r: r.delta_f1),
155         "mean_precision_on": _mean(lambda r: r.on.precision),
156         "mean_precision_off": _mean(lambda r: r.off.precision),
157         "mean_recall_on": _mean(lambda r: r.on.recall),
158         "mean_recall_off": _mean(lambda r: r.off.recall),
159         "mean_seg_ratio_on": _mean(lambda r: r.seg_ratio_on),
160         "mean_seg_ratio_off": _mean(lambda r: r.seg_ratio_off),
161         "wins": wins,
162         "losses": losses,
163     }
164
165
166 def format_table(
167     results: List[ServedVideoResult], agg: Dict[str, Any], iou: float
168 ) -> str:
169     lines = [
170         f"=== SERVED Gate-A verification (production FusionSegmenter path, IoU>={iou})
===",
171         f" {'video':<24s} {'gt':>3s} {'F1_off':>7s} {'F1_on':>7s} {'?F1':>7s} "
172         f"{'segR_off':>9s} {'segR_on':>8s} {'hand_off':>9s} {'hand_on':>8s}",
173     ]
174     for r in results:
175         lines.append(
176             f" {r.name:<24s} {r.num_gt:>3d} {r.off.f1:>7.3f} {r.on.f1:>7.3f} "
177             f"{r.delta_f1:>7.3f} {r.seg_ratio_off:>9.2f} {r.seg_ratio_on:>8.2f} "
178             f"{r.n_handoff_off:>9d} {r.n_handoff_on:>8d}"
179         )
180     lines += [
181         f" {'-' * 92}",
182         f" MEAN F1 {agg['mean_f1_off']:.3f} (off) -> {agg['mean_f1_on']:.3f} (on) "
183         f"?F1={agg['mean_delta_f1']+.3f} "
184         f"P {agg['mean_precision_off']:.3f}->{agg['mean_precision_on']:.3f} "
185         f"R {agg['mean_recall_off']:.3f}->{agg['mean_recall_on']:.3f}",
186         f" over-seg ratio {agg['mean_seg_ratio_off']:.2f} (off) -> "
187         f"{agg['mean_seg_ratio_on']:.2f} (on) | wins {agg['wins']}/{agg['num_videos']}
",
188         f"(losses {agg['losses']})",
189     ]
190     return "\n".join(lines)
191
192
193 def run(
194     manifest_path: str, iou: float = 0.5
195 ) -> Tuple[List[ServedVideoResult], Dict[str, Any]]:

```

```

196         with open(manifest_path, encoding="utf-8") as f:
197             specs = json.load(f)
198             results = evaluate_manifest(specs, iou=iou)
199             return results, aggregate(results)
200
201
202     def main(argv: Optional[List[str]] = None) -> None:
203         ap = argparse.ArgumentParser(
204             description="Verify served Gate-A on real footage via the production
FusionSegmenter path."
205         )
206         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
207         ap.add_argument("--iou", type=float, default=0.5)
208         args = ap.parse_args(argv)
209         results, agg = run(args.manifest, iou=args.iou)
210         print(format_table(results, agg, args.iou))
211
212
213     if __name__ == "__main__":
214         main()
215

```

4. user (2026-06-29T17:01:45.607Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13    torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14    model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15    via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16    - ``select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19    **Persisted candidates remain the full superset** regardless, so the offline
experiment
20    harness can re-score any variant.
21
22    Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23    nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24    and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25    segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26    experiment harness (EXPERIMENT_HARNESS.md), never silently.
27    """
28
29    from typing import Any, Dict, List, Optional, Tuple
30
31    from backend.config.models import FusionConfig, IndexingConfig

```

```

32     from backend.pipeline.detectors import fusion_features as ff
33     from backend.pipeline.detectors import rally_gate
34     from backend.pipeline.detectors.base import DetectorRunner
35     from backend.pipeline.detectors.tracknet_runner import TrackNetConfig, TrackNetRunner
36     from backend.pipeline.detectors.wasb_runner import WasbConfig, WasbRunner
37     from backend.pipeline.segmenters.base import segmenter_registry
38     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
39
40
41     class FusionSegmenter(TrajectoryHybridSegmenter):
42         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
43 B)."""
44         family = "fusion"
45         display_label = "Fusion"
46
47         def __init__(self, db: Any, config: Any):
48             super().__init__(db, config)
49             # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
50             self._person: Optional[Any] = None
51             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
52             # over the whole trajectory once per candidate window (it depends only on
`points`).
53             self._axis_cache_key: Optional[int] = None
54             self._axis_cache: Optional[tuple] = None
55
56         @property
57         def name(self) -> str:
58             return "fusion_hybrid"
59
60         @property
61         def description(self) -> str:
62             return (
63                 "Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
64                 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them, and
the "
65                 "AI provider sets semantic boundaries."
66             )
67
68         def _build_runner(
69             self, idx_cfg: IndexingConfig
70         ) -> Tuple[DetectorRunner, Dict[str, Any]]:
71             if isinstance(idx_cfg, dict):
72                 idx_cfg = IndexingConfig(**idx_cfg)
73             substrate = idx_cfg.fusion.substrate
74             if substrate == "tracknet":
75                 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
76                 return TrackNetRunner(cfg), {
77                     "weights_path": cfg.weights_path,
78                     "queue_length": cfg.queue_length,
79                     "fusion_substrate": "tracknet",
80                 }
81             wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
82             return WasbRunner(wcfg), {
83                 "weights_path": wcfg.weights_path,
84                 "fusion_substrate": "wasb",
85             }
86
87         def _build_native_runner(
88             self, idx_cfg: IndexingConfig
89         ) -> Tuple[DetectorRunner, Dict[str, Any]]:
90             # Native (no-WSL) path supports the WASB substrate only ? there is no native

```

```

TrackNet
91         # runner yet. Fusion's default substrate is WASB, so this covers the common
case.
92         substrate = idx_cfg.fusion.substrate
93         if substrate != "wasb":
94             raise NotImplementedError(
95                 "detector_impl='native' supports the WASB substrate only ? set "
96                 "indexing.fusion.substrate='wasb' (no native TrackNet runner yet).")
97         )
98         from backend.pipeline.detectors.native_wasb_runner import (
99             NativeWasbConfig,
100             NativeWasbRunner,
101         )
102
103         cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
104         return NativeWasbRunner(cfg), {
105             "weights_path": cfg.weights_path,
106             "detector_impl": "native",
107             "fusion_substrate": "wasb",
108             "device": cfg.device,
109         }
110
111     def _enrich_candidate_stats(
112         self,
113         cw: Dict[str, Any],
114         points: List[Any],
115         fps: float,
116         frame_width: float,
117         video_path: str,
118         idx_cfg: IndexingConfig,
119     ) -> Dict[str, Any]:
120         stats = super()._enrich_candidate_stats(
121             cw, points, fps, frame_width, video_path, idx_cfg
122         )
123         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
124         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
125         # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
126         gated = rally_gate.gate_windows(
127             points,
128             [(cw["start"], cw["end"])],
129             fps,
130             min_crossings=0,
131             estimate=self._axis_estimate(points),
132         )
133         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
134
135         fcfg = idx_cfg.fusion
136         if fcfg.cue_flags.get("person", False):
137             stats.update(
138                 self._person_features(cw, points, fps, frame_width, video_path, fcfg)
139             )
140         return stats
141
142     # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
143     # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
144     # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
145     # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
146     # self._person + windowed detections_per_frame. Next: actual harness run + lift

```

```

measurement
147         # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
148         # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
149
150     def _axis_estimate(self, points: List[Any]) -> tuple:
151         """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
152         computed once per video, not once per candidate window (it depends only on
points)."""
153         key = id(points)
154         if self._axis_cache_key != key or self._axis_cache is None:
155             self._axis_cache_key = key
156             self._axis_cache = rally_gate.estimate_play_axis(points)
157         est = self._axis_cache
158         assert est is not None
159         return est
160
161     def _person_features(
162         self,
163         cw: Dict[str, Any],
164         points: List[Any],
165         fps: float,
166         frame_width: float,
167         video_path: str,
168         fcfg: "FusionConfig",
169     ) -> Dict[str, float]:
170         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
171         frame range and compute the scale/translation-invariant person features. Lazily
builds
172         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
173         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
174         from backend.pipeline.detectors.person_detector import PersonDetector
175
176         if self._person is None:
177             self._person = PersonDetector(self.config)
178
179         frame_skip = self.config.indexing.yolo_frame_skip
180         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
181         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
182         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
183         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
184         fw = det["frame_width"] or frame_width
185         fh = det["frame_height"]
186         # If we can't trust the frame dims, every normalised feature would silently
collapse
187         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
188         if fw <= 0 or fh <= 0:
189             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
190
191         poly = fcfg.court_polygon
192         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
193         net = (fcfg.net_axis, float(fcfg.net_position))
194         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
195         return ff.compute_person_features(
196             det["frames"], shuttle_in, fw, fh, court_polygon=polygon, net=net
197         )
198
199     def _select_for_handoff(
200         self,

```



```

201         windows: List[Dict[str, Any]],
202         window_stats: List[Dict[str, Any]],
203         idx_cfg: IndexingConfig,
204     ) -> List[int]:
205         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
206         (0) keep every window ? identical to the substrate. Persisted candidates are
207         untouched (full superset).
208
209         NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
210         thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
211         where no segment clears ``min_crossings``), the gate falls back to the full
ungated
212         set rather than emit a video with zero rallies. This makes Gate A/B strictly
213         *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
214         the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
215         below it always wins on an otherwise-empty video. Observed on real footage
216         (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
217         gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
218         fcfg = idx_cfg.fusion
219         min_crossings = int(fcfg.min_crossings)
220         min_players = int(fcfg.min_players)
221         person_on = bool(fcfg.cue_flags.get("person", False))
222
223         keep: List[int] = []
224         for i, st in enumerate(window_stats):
225             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
226                 continue
227             if (
228                 person_on
229                 and min_players > 0
230                 and st.get("players_in_court", 0.0) < min_players
231             ):
232                 continue
233             keep.append(i)
234
235         if not keep and window_stats:
236             # Gating emptied a non-empty video ? fall back to the ungated set.
237             return list(range(len(window_stats)))
238         return keep
239
240
241     segmenter_registry.register(FusionSegmenter)
242

```

5. user (2026-06-29T17:01:46.197Z)

```

1     """SERVED Gate-A regression litmus ? the *production* FusionSegmenter path on real
footage.
2
3     The flagged gap (#146): the served default flip (``default_segmenter=fusion_hybrid`` +
4     ``min_crossings=2``) is **mock-tested only** on real video (``test_fusion_hybrid.py``
drives a
5     MagicMock runner). The offline ablation litmus
6     (``test_ablation.test_litmus_reproduces_gate_a``)
7     validates the Gate-A *signal* ? but on the **permissive proposal windowing**
8     (``distill_local.PROPOSAL``), NOT the production operating point. This module closes the
gap: it
9     runs the real ``FusionSegmenter`` decision seams (``enrich_candidate_stats`` net-
crossings +

```

```

9     ``select_for_handoff`` Gate A) over windows from the **production**
10     ``trajectory_to_action_windows`` config, on the cached golden WASB trajectories, and
asserts
11     the served path's measured behaviour.
12
13     GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips
cleanly in
14     CI where the golden trajectories are absent; runs on the owner box where they are
present, over
15     **whatever golden clips are currently on disk** (``golden_specs`` requires >= 6).
16
17     CORPUS NOTE ? re-grounded 2026-06-22 (the golden set is mutable and grows over time;
this is the
18     SECOND re-grounding ? #236 pinned the 2026-06-18 9-clip snapshot, which the corpus has
since
19     outgrown). The original 6 *single-court* golden clips this litmus was first calibrated
on
20     (2026-06-14) have been retired from the box ? their labels are gone ? so the present
golden set is
21     the **15 multicourt clips** now on disk (mahadevpura_1/2/singles, GX0101xx, GX0x0094,
22     Badminton_BXH_2, Boxhill_Doubles, adarsh_avi_singles, kushagra_singles,
largetest_doubles, gbaaddy,
23     testlarge_short). On that corpus the served path is **recall-starved**: WASB tracks a
single shuttle
24     while several courts rally at once, so ``trajectory_to_action_windows`` emits FEWER
windows than
25     there are rallies (served over-seg ratio < 1, i.e. under-segmentation). Because Gate A
only ever
26     DROPS windows, on this under-segmented corpus it is mildly **F1-negative** (mean served
?F1 ?
27     -0.023), not the ~neutral -0.008 the original 6 showed. Under the permissive PROPOSAL
windowing the
28     same gate is ~F1-neutral (?F1 ? +0.03) while still cutting over-segmentation
(1.36?0.79).
29
30     The canonical **+0.074 / 6-of-6 / p=0.031** Gate-A win was a property of the
*original-6* corpus
31     and is NOT reproduced here (those clips are no longer on the box); it is preserved in
the archive
32     (``docs/archives/quality-iterations/2026-06-first-ab-experiment/``). These tests now pin
the
33     **present-corpus** behaviour so a future windowing/corpus change that silently shifts it
trips the
34     litmus. The pinned magnitudes are a date-stamped snapshot of the present golden set and
will need
35     re-measurement as the corpus grows.
36     ""
37
38     from __future__ import annotations
39
40     import json
41     import os
42
43     import pytest
44
45     # cv2 is imported transitively by trajectory_hybrid (the served engine). On a box
without it
46     # (CI), skip the whole module rather than erroring at import.
47     pytest.importorskip("cv2")
48
49     from backend.eval import segmentation_metrics as _segm # noqa: E402
50     from backend.eval import served_gate_a as sga # noqa: E402
51     from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
52     from backend.eval.metrics import score # noqa: E402
53     from backend.pipeline.detectors import rally_gate # noqa: E402

```

```

54     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
55
56     # The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a
fresh
57     # worktree / CI). Resolve it relative to this repo root first, with an env override for
an
58     # out-of-tree checkout.
59     _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
60     MANIFEST = os.environ.get(
61         "SERVED_GATE_A_MANIFEST",
62         os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
63     )
64
65
66     def _golden_specs():
67         """Return the manifest specs whose trajectory + label files are all present, else
68         []."""
69         if not os.path.isfile(MANIFEST):
70             return []
71         with open(MANIFEST, encoding="utf-8") as f:
72             specs = json.load(f)
73             ok = [
74                 s
75                 for s in specs
76                 if os.path.isfile(str(s.get("trajectory", "")))
77                 and os.path.isfile(str(s.get("labels", "")))
78             ]
79             return ok if len(ok) >= 6 else []
80
81     _SPECS = _golden_specs()
82     _skip = pytest.mark.skipif(
83         not _SPECS,
84         reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
85     )
86
87
88     # ----- served-path
litmus
89
90
91     @_skip
92     def test_served_path_runs_on_all_golden_videos():
93         """The production FusionSegmenter decision path runs end-to-end (no GPU/Gemini) on
EVERY
94         present golden trajectory and yields a handoff window list for each, for both gate
settings.
95         Asserts against the present corpus count (>= 6), not a stale fixed 6 ? the golden
set grows."""
96         results = sga.evaluate_manifest(_SPECS)
97         assert len(results) == len(_SPECS)
98         assert len(results) >= 6
99         for r in results:
100             assert r.num_gt > 0
101             assert (
102                 r.n_handoff_off >= r.n_handoff_on
103             ) # Gate A only ever drops windows, never adds
104
105
106     @_skip
107     def test_served_gate_a_does_not_materially_regress_f1():
108         """SERVED no-material-regression guard: on the production windowing Gate A must not
MATERIALLY
109         hurt F1. On the present recall-starved multicourt corpus the served path is under-
segmented

```

```

110         (over-seg ratio < 1), so Gate A ? which only DROPS windows ? trims a few borderline-
true
111         windows and is mildly negative, NOT the ~neutral it was on the original single-court
6. We pin
112         a band that admits this measured mild loss but trips on a material regression."""
113         results = sga.evaluate_manifest(_SPECS)
114         agg = sga.aggregate(results)
115         # Honest measured value (2026-06-22, present 15-clip multicourt golden set): mean
served
116         # ?F1 ? -0.023 (was ? -0.034 on the 9-clip / ? -0.008 on the retired original-6).
Band admits
117         # the mild recall-starved loss while still tripping if Gate A materially destroys
served F1.
118         assert -0.06 <= agg["mean_delta_f1"] <= 0.03, agg
119         # And Gate A must never collapse served F1 to ~half the OFF baseline (measured ratio
? 0.90).
120         assert agg["mean_f1_on"] >= 0.5 * agg["mean_f1_off"] - 1e-9, agg
121
122
123     @_skip
124     def test_served_gate_a_does_not_increase_over_segmentation():
125         """Gate A only ever DROPS windows, so the served over-seg ratio must not rise when
it's on
126         (the direction-of-effect invariant ? even where the absolute lift is ~neutral)."""
127         results = sga.evaluate_manifest(_SPECS)
128         agg = sga.aggregate(results)
129         assert agg["mean_seg_ratio_on"] <= agg["mean_seg_ratio_off"] + 1e-9, agg
130         # Per-video too: the gate is monotone (kept windows ? all windows).
131         for r in results:
132             assert r.seg_ratio_on <= r.seg_ratio_off + 1e-9, r
133
134
135     @_skip
136     def test_served_uses_production_windowing_operating_point():
137         """Pin the served operating point to config.json's indexing defaults ? if someone
retunes
138         production windowing, this litmus's premise (and the divergence it documents) is
revisited
139         deliberately, not silently."""
140         assert sga.PROD_VELOCITY_THRESH == 0.02
141         assert sga.PROD_MERGE_GAP == 2.0
142         assert sga.PROD_MIN_WINDOW == 2.0
143
144
145     # ----- Gate A under the permissive PROPOSAL
windowing
146
147     # distill_local.PROPOSAL ? the permissive, recall-oriented candidate windowing the
OFFLINE
148     # ablation/fusion_compare harness builds on (low velocity thresh, short merge/min-dur ?
many
149     # small windows). On the original-6 single-court corpus the Gate-A lift here was +0.074
(6/6,
150     # p=0.031; archived). On the present multicourt corpus that lift is GONE ? the gate is
~F1-neutral
151     # but still cuts over-segmentation. We pin the present-corpus behaviour, not the retired
+0.074.
152     _PROPOSAL = (0.005, 1.0, 0.5)
153
154
155     @_skip
156     def test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_over_seg():
157         """CONTRAST pin: under the OFFLINE harness's permissive proposal windowing, the SAME
158         ``rally_gate`` net-crossing gate the served seam uses behaves DIFFERENTLY than under
the

```

```

159     production windowing. The durable invariant (any corpus): the gate reduces over-
segmentation
160     (seg_on < seg_off), since it only drops windows. On the present multicourt corpus it
is
161     ~F1-neutral (?F1 ? -0.001, was +0.074 on the now-retired original-6) ? proving the
served
162     behaviour is a property of the operating point + corpus, not a broken gate."""
163     d_on, d_off, seg_on, seg_off = [], [], [], []
164     for s in _SPECS:
165         fps, fw = float(s["fps"]), float(s["frame_width"])
166         pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
167         wins = TrackNetRunner.trajectory_to_action_windows(
168             pts,
169             fps=fps,
170             frame_width=fw,
171             velocity_thresh=_PROPOSAL[0],
172             merge_gap=_PROPOSAL[1],
173             min_window_duration=_PROPOSAL[2],
174         )
175         iv = [(w["start"], w["end"]) for w in wins]
176         est = rally_gate.estimate_play_axis(pts)
177         gated = rally_gate.gate_windows(pts, iv, fps, min_crossings=0, estimate=est)
178         on = [(g.start, g.end) for g in gated if g.crossings >= 2]
179         gts = load_golden_gts(s["labels"])
180         d_on.append(score(on, gts).f1)
181         d_off.append(score(iv, gts).f1)
182         seg_on.append(_segm.segment_count_ratio(on, gts))
183         seg_off.append(_segm.segment_count_ratio(iv, gts))
184     n = len(d_on)
185     mean_delta = (sum(d_on) - sum(d_off)) / n
186     mean_seg_off, mean_seg_on = sum(seg_off) / n, sum(seg_on) / n
187     # Durable direction-of-effect (corpus-independent): the gate only drops windows, so
under the
188     # over-segmenting proposal windowing it lowers the over-seg ratio.
189     assert mean_seg_on < mean_seg_off, (mean_seg_on, mean_seg_off)
190     # Honest measured snapshot (2026-06-22, present 15-clip multicourt set): mean ?F1 ?
+0.03,
191     # over-seg 1.36 ? 0.79 (was -0.001 / 1.16 ? 0.64 on the 9-clip; +0.074 / 1.68 ? 1.01
on the
192     # retired original-6). ~F1-neutral band:
193     assert mean_delta == pytest.approx(0.03, abs=0.03), mean_delta
194     assert mean_seg_off == pytest.approx(1.36, abs=0.10), mean_seg_off
195     assert mean_seg_on == pytest.approx(0.79, abs=0.10), mean_seg_on
196

```

6. assistant (2026-06-29T17:01:48.400Z)

Now let me look at the production `process\_video` path and the config models to confirm what type the production code passes as `idx\_cfg`.

7. user (2026-06-29T17:01:49.825Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7     healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8     -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").

```

```

14 - ``name`` / ``description`` properties (the registry contract).
15 - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16     detection_params extras such as weights_path).
17
18 This base is deliberately NOT registered; only concrete subclasses register.
19 """
20
21 import concurrent.futures
22 import logging
23 import os
24 import time
25 from typing import Any, Dict, List, Optional, Tuple
26
27 import cv2
28
29 from backend.config.models import IndexingConfig
30 from backend.pipeline.detectors.base import DetectorRunner
31 from backend.pipeline.segmenters.base import BasePlaySegmenter
32 from backend.providers import provider_registry
33 from backend.sports import sport_registry
34 from backend.utils.paths import output_base
35 from backend.utils.video import extract_video_chunk, get_video_duration
36
37 logger = logging.getLogger(__name__)
38
39
40 def _fmt_ts(seconds: float) -> str:
41     seconds = max(0, int(seconds))
42     h, rem = divmod(seconds, 3600)
43     m, s = divmod(rem, 60)
44     return f"{h:02d}:{m:02d}:{s:02d}"
45
46
47 class TrajectoryHybridSegmenter(BasePlaySegmenter):
48     """Trajectory-detector hybrid: precise candidate rallies + AI semantic
49 boundaries."""
50     #: config section under `indexing` (velocity_threshold lives there), the
51     #: output subdir for trajectories, and the metadata detector-tag prefix.
52     family: str = ""
53     #: human-readable label used in log lines.
54     display_label: str = ""
55
56     def _build_runner(
57         self, idx_cfg: IndexingConfig
58     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
59         """Return (runner, detector-specific detection_params extras) for the default
60 (WSL) path."""
61         raise NotImplementedError
62
63     def _build_native_runner(
64         self, idx_cfg: IndexingConfig
65     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
66         """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
67 with a
68 native runner override this; the base refuses with a clear message (M3.5: WASB
69 only)."""
70         raise NotImplementedError(
71             f"detector_impl='native' is not supported for the "
72             f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
73 has a "
74             f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'."
75         )
76
77     def _resolve_runner(

```

```

74         self, idx_cfg: IndexingConfig
75     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
76         """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
77
78         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
79         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
80         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
81         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
82         ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
83         (the WSL runner) so the default production path is unchanged
84         (``docs/DECOUPLED_COMPUTE/``).
85         """
86         # Safety net: accept raw dicts from legacy / test callers.
87         if isinstance(idx_cfg, dict):
88             idx_cfg = IndexingConfig(**idx_cfg)
89         impl = (
90             os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.detector_impl or "auto"
91         ).lower()
92         if impl == "stub":
93             from backend.pipeline.detectors.stub_runner import (
94                 StubConfig,
95                 StubDetectorRunner,
96             )
97
98             logger.info(
99                 "%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
100                 self.display_label or "trajectory",
101             )
102             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {
103                 "detector_impl": "stub"
104             }
105         if impl in ("native", "native_wasb", "wasb_native"):
106             logger.info(
107                 "%s: detector_impl=native ? Linux-native runner (no WSL).",
108                 self.display_label or "trajectory",
109             )
110             return self._build_native_runner(idx_cfg)
111         return self._build_runner(idx_cfg)
112
113     @staticmethod
114     def _probe_fps_width(video_path: str) -> tuple:
115         """Return (fps, frame_width) for a video, with sensible fallbacks."""
116         cap = cv2.VideoCapture(video_path)
117         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
118         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
119         cap.release()
120         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
121
122     @staticmethod
123     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
124         """Provider-reported exchange count, else a duration-based estimate.
125
126         One canonical implementation ? the twins had drifted (wasb relied on
127         ``int(None)`` raising into the fallback; same result, by accident).
128         """
129         shot_count = segment.get("shuttle_exchange_count")
130         if shot_count is None:
131             return max(3, int(duration / 0.8))
132         try:
133             return int(shot_count)

```

```

134         except (ValueError, TypeError):
135             return max(3, int(duration / 0.8))
136
137     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
138     -----
139     def _enrich_candidate_stats(
140         self,
141         cw: Dict[str, Any],
142         points: List[Any],
143         fps: float,
144         frame_width: float,
145         video_path: str,
146         idx_cfg: "IndexingConfig",
147     ) -> Dict[str, Any]:
148         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
149         BASE
150         returns exactly the 4 motion keys, so the trajectory twins persist byte-
151         identical
152         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
153         features). ``points`` are the whole-video trajectory points
154         (TrajectoryPoint)."""
155         return {
156             "peak_velocity": cw["peak_velocity"],
157             "mean_velocity": cw["mean_velocity"],
158             "active_frame_count": cw["active_frame_count"],
159             "track_id_count": cw["track_id_count"],
160         }
161
162     def _select_for_handoff(
163         self,
164         windows: List[Dict[str, Any]],
165         window_stats: List[Dict[str, Any]],
166         idx_cfg: "IndexingConfig",
167     ) -> List[int]:
168         """Indices of the candidate windows that proceed to the AI handoff (and thus may
169         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
170         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
171         Persisted candidates are NOT affected ? they remain the full superset for
172         offline
173         re-scoring."""
174         return list(range(len(windows)))
175
176     def process_video( # type: ignore[override]
177         self,
178         video_id: str,
179         video_path: str,
180         player_pool: Optional[List[str]] = None,
181         max_frames: Optional[int] = None,
182     ) -> List[Dict[str, Any]]:
183         # override-ignore: the ABC declares the Result contract (Success|Failure)
184         # but every live segmenter still returns a bare list ? the documented
185         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
186         label = self.display_label
187         # --- Sport profile + AI provider wiring (identical across segmenters) ---
188         sport_name = self.config.sport
189         try:
190             sport_profile = sport_registry.get(sport_name)
191         except KeyError as e:
192             logger.error(str(e))
193             return []
194
195         idx_cfg = self.config.indexing
196         skip_ai = bool(idx_cfg.skip_ai_handoff)
197         provider_name = self.config.ai_provider
198         if skip_ai:

```



```

194         model_name = "local-noai"
195         logger.info(
196             "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
197             "be emitted as rallies; no %s handoff, no API key needed.",
198             label,
199             provider_name,
200         )
201     else:
202         model_name = self.config.ai_model
203         api_key_env_var = f"{provider_name.upper()}_API_KEY"
204         api_key = os.environ.get(api_key_env_var)
205         if not api_key:
206             from backend.pipeline.segmenters._keyhint import api_key_hint
207
208             logger.error(
209                 f"{api_key_env_var} environment variable is not set! "
210                 f"To run the {provider_name} handoff, set your API key. "
211                 + api_key_hint(api_key_env_var)
212             )
213             return []
214         try:
215             provider = provider_registry.get(
216                 provider_name, api_key=api_key, model_name=model_name
217             )
218         except KeyError as e:
219             logger.error(str(e))
220             return []
221
222     video_duration = get_video_duration(video_path)
223     if video_duration <= 0:
224         logger.error("Invalid video duration.")
225         return []
226     fps, frame_width = self._probe_fps_width(video_path)
227
228     # --- Runner config + windowing thresholds ---
229     # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
230     # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
231     # it delegates to the subclass _build_runner (the real WSL/native runner).
232     try:
233         runner, runner_params = self._resolve_runner(idx_cfg)
234     except NotImplementedError as e:
235         # e.g. detector_impl="native" for a family without a native runner
236         # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
237         # crashing.
238         logger.error("%s: %s", label, e)
239         return []
240
241     velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
242     merge_gap = idx_cfg.dead_time_merge_gap
243     min_window_duration = idx_cfg.min_action_window_duration
244     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
245     IndexingConfig.
246     max_window_duration = idx_cfg.max_window_duration
247     window_min_split_gap = idx_cfg.window_min_split_gap
248     window_hard_cap = idx_cfg.window_hard_cap
249     window_inactivity_end = idx_cfg.window_inactivity_end
250     inactivity_rally_thresh = idx_cfg.inactivity_rally_thresh
251     inactivity_min_density = idx_cfg.inactivity_min_density
252     inactivity_temporal_density = idx_cfg.inactivity_temporal_density
253     window_blob_fallback = idx_cfg.window_blob_fallback
254     window_blob_factor = idx_cfg.window_blob_factor
255     handoff_padding = idx_cfg.ai_handoff_padding
256     ai_max_workers = idx_cfg.ai_max_workers
257     log_candidates = idx_cfg.log_yolo_candidates

```

```

256     store_candidates = idx_cfg.store_yolo_candidates
257
258     detection_params = {
259         "detector": self.name,
260         "velocity_thresh": velocity_thresh,
261         "merge_gap": merge_gap,
262         "min_action_window_duration": min_window_duration,
263         **runner_params,
264     }
265
266     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
267     ok, msg = runner.healthcheck()
268     if not ok:
269         logger.error("%s detector environment not ready: %s", label, msg)
270         return []
271     logger.info("%s detector env OK: %s", label, msg)
272
273     # --- Phase 2: trajectory -> candidate rally windows ---
274     # Trajectory detectors process the whole video in one pass (they carry
275     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
276     t_start = time.time()
277     out_dir = os.path.join(output_base(self.config), self.family)
278     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
279     if not csv_path:
280         logger.error("%s produced no trajectory; aborting.", label)
281         return []
282
283     points = runner.parse_trajectory_csv(csv_path)
284     logger.info(
285         "Parsed %d trajectory points (%d visible).",
286         len(points),
287         sum(1 for p in points if p.visible),
288     )
289
290     all_candidate_windows = runner.trajectory_to_action_windows(
291         points,
292         fps=fps,
293         frame_width=frame_width,
294         chunk_start=0.0,
295         velocity_thresh=velocity_thresh,
296         merge_gap=merge_gap,
297         min_window_duration=min_window_duration,
298         chunk_index=0,
299         max_window_duration=max_window_duration,
300         min_split_gap>window_min_split_gap,
301         window_hard_cap>window_hard_cap,
302         window_inactivity_end>window_inactivity_end,
303         inactivity_rally_thresh=inactivity_rally_thresh,
304         inactivity_min_density=inactivity_min_density,
305         inactivity_temporal_density=inactivity_temporal_density,
306         window_blob_fallback>window_blob_fallback,
307         window_blob_factor>window_blob_factor,
308     )
309     logger.info(
310         "Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
311         label,
312         time.time() - t_start,
313         len(all_candidate_windows),
314     )
315
316     if log_candidates:
317         for cw in all_candidate_windows:
318             logger.info(
319                 f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
320                 f"({cw['end'] - cw['start']:.1f}s) |"

```

```

frames={cw['active_frame_count']} "
321         f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}"
322     )
323
324     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
325     # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
326     # persistence and the handoff gate below.
327     window_stats = [
328         self._enrich_candidate_stats(
329             cw, points, fps, frame_width, video_path, idx_cfg
330         )
331         for cw in all_candidate_windows
332     ]
333
334     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
335     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
336     if store_candidates:
337         for i, cw in enumerate(all_candidate_windows):
338             candidate_ids[i] = self.db.add_candidate_segment(
339                 video_id,
340                 detector=self.name,
341                 start_time=cw["start"],
342                 end_time=cw["end"],
343                 chunk_index=cw["chunk_index"],
344                 confidence=min(1.0, cw["peak_velocity"]),
345                 stats=window_stats[i],
346                 detection_params=detection_params,
347             )
348
349     if not all_candidate_windows:
350         return []
351
352     # --- Phase 3: AI provider handoff over padded candidate clips ---
353     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
354     # candidates above stay the full superset ? only the served play_segments are
gated.
355     handoff_indices = self._select_for_handoff(
356         all_candidate_windows, window_stats, idx_cfg
357     )
358
359     ai_segments: List[dict] = []
360     if skip_ai:
361         # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
362         # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
363         # falls back to the duration-based estimate (no AI exchange count).
364         ai_segments = [
365             {
366                 "start_time": all_candidate_windows[wi]["start"],
367                 "end_time": all_candidate_windows[wi]["end"],
368                 "shuttle_exchange_count": None,
369                 "shot_count_confidence": "LocalNoAI",
370                 "_candidate_id": candidate_ids[wi],
371             }
372             for wi in handoff_indices
373         ]
374         logger.info(
375             "Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies
"
376             "(no AI handoff).",
377             len(ai_segments),

```

```

378         )
379     else:
380         handoff_dir = os.path.join(
381             output_base(self.config), f"chunks_{provider_name}_handoff"
382         )
383         os.makedirs(handoff_dir, exist_ok=True)
384         logger.info(
385             "Phase 3: %s validation on %d candidate windows...",
386             provider_name,
387             len(handoff_indices),
388         )
389
390     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
391         w_start = max(0, window["start"] - handoff_padding)
392         w_end = min(video_duration, window["end"] + handoff_padding)
393         w_dur = w_end - w_start
394         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
395         if not extract_video_chunk(
396             video_path, w_start, w_dur, w_path, reencode=True
397         ):
398             return []
399         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
400         segments, metrics = provider.analyze_video(
401             w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
402         )
403         # B2: a provider failure returns [] with metrics["error"] set ? that is
404         # "we don't know", NOT "no rallies in this window". Surface it loudly so
405         # a
406         # partial run isn't mistaken for a complete one (mirrors the gemini
407         segmenter).
408         if metrics.get("error"):
409             logger.warning(
410                 "[Handoff %d] provider error ? window %d skipped: %s",
411                 wi + 1,
412                 wi,
413                 metrics["error"],
414             )
415         valid = []
416         for seg in segments:
417             try:
418                 seg["start_time"] = float(seg["start_time"]) + w_start
419                 seg["end_time"] = float(seg["end_time"]) + w_start
420                 seg["_candidate_id"] = candidate_ids[wi]
421                 valid.append(seg)
422             except (ValueError, TypeError, KeyError) as e:
423                 logger.warning(
424                     "Dropping segment with unparseable timestamps: %s ? %s",
425                     seg,
426                     e,
427                 )
428         try:
429             os.remove(w_path)
430         except OSError:
431             pass
432         return valid
433
434     with concurrent.futures.ThreadPoolExecutor(
435         max_workers=ai_max_workers
436     ) as executor:
437         futures = [
438             executor.submit(
439                 process_candidate_window, wi, all_candidate_windows[wi]
440             )
441             for wi in handoff_indices
442         ]

```

```

441         for future in futures:
442             ai_segments.extend(future.result())
443     try:
444         os.rmdir(handoff_dir)
445     except OSError:
446         pass
447     logger.info(
448         "Phase 3 completed. AI confirmed %d semantic play segments.",
449         len(ai_segments),
450     )
451
452     # --- Phase 4: DB population + candidate linking ---
453     segments_data = []
454     for segment in ai_segments:
455         try:
456             start = float(segment["start_time"])
457             end = float(segment["end_time"])
458             dur = end - start
459             metadata = {
460                 "shot_count": self._shot_count_from(segment, dur),
461                 "shot_count_confidence": segment.get(
462                     "shot_count_confidence", "Unknown"
463                 ),
464                 "detector": f"{self.family}-hybrid-{model_name}",
465             }
466             segment_id = self.db.add_play_segment(
467                 video_id, start, end, None, None, metadata
468             )
469             if segment_id:
470                 candidate_id = segment.get("_candidate_id")
471                 if candidate_id is not None:
472                     self.db.link_candidate_to_segment(candidate_id, segment_id)
473                 segments_data.append(
474                     {
475                         "id": segment_id,
476                         "start_time": start,
477                         "end_time": end,
478                         "duration": dur,
479                         "metadata": metadata,
480                     }
481                 )
482         except Exception as e:
483             logger.warning("Skipping invalid segment struct: %s", e)
484
485     return segments_data
486

```

8. user (2026-06-29T17:01:50.762Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     Unknown / typo'd config keys are rejected at load time via extra="forbid".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16

```

```

17
18 class ValidationRelevanceConfig(BaseModel):
19     court_green_lower: list[int] = Field(default=[35, 40, 40])
20     court_green_upper: list[int] = Field(default=[85, 255, 255])
21     court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24 class ValidationConfig(BaseModel):
25     min_resolution_width: int = 1280
26     min_resolution_height: int = 720
27     min_fps: float = 29.9
28     min_blur_threshold: float = 20.0
29     max_scene_cuts_per_minute: float = 0.5
30     pts_max_gap_seconds: float = Field(
31         default=10.0,
32         ge=0.0,
33         description=(
34             "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
35             "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
36             "a gap >10s suggests edited/missing content. Mobile phones, screen "
37             "recordings, and other encoders may use different GOP sizes ? tune "
38             "this threshold for your recording device."
39         ),
40     )
41     relevance: ValidationRelevanceConfig = Field(
42         default_factory=ValidationRelevanceConfig
43     )
44
45
46 class TrackNetConfig(BaseModel):
47     wsl_distro: str = "Ubuntu"
48     repo_dir: str = "~/models/TrackNetV4"
49     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50     conda_env: str = "TrackNetV4"
51     weights_path: str = ""
52     queue_length: int = 5
53     stage_in_wsl: bool = True
54     wsl_stage_dir: str = "~/clips"
55     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56     velocity_threshold: float = 0.02
57     # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58     # decoded frame cache) are deleted automatically ? they are pure intermediates,
59     # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60     # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61     keep_frames: bool = False
62     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
63     # 0 = disabled.
64     min_free_gb: float = 0.0
65
66
67 class WasbIndexConfig(BaseModel):
68     """Typed config for the live WASB shuttle substrate (indexing.wasb).
69
70     Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
71     these knobs actually take effect ? previously this whole block was silently dropped
72     because nested config sections defaulted to extra='ignore'.
73     """
74
75     wsl_distro: str = "Ubuntu"
76     repo_dir: str = "~/models/WASB-SBDT"
77     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
78     conda_env: str = "wasb"
79     weights_path: str = (
80         "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
81     )

```

```

82     sport: str = "badminton"
83     wsl_stage_dir: str = "~/clips"
84     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
85     velocity_threshold: float = 0.02
86     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
87     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
88     # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
89     # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
90     device: str = "cuda" # auto | cuda | mps | cpu
91     python_bin: str = (
92         "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
93     )
94     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
95     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
96     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
97     keep_frames: bool = False
98     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
99     min_free_gb: float = 0.0
100     # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
101     # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
102     # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
103     # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
104     # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
105     # Tri-state:
106     #     None      = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
107     #     behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
108     #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
109     stream_video: Optional[bool] = None
110
111
112     class FusionConfig(BaseModel):
113         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
114
115         Every default reproduces control behaviour: the fusion segmenter persists the
116         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
117         true ? the person-cue features, but it does NOT gate the served handoff unless a
118         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
119         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
120         supplied ? off-court persons (spectators / adjacent courts) are excluded.
121         """
122
123         # Which trajectory substrate seeds the windows (shuttle stays primary).
124         substrate: Literal["wasb", "tracknet"] = "wasb"
125         # Windowing velocity threshold for the fusion family (the engine reads
126         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
127         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
128         velocity_threshold: float = 0.02
129         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly

```

```

130         # enabled ? so the default path never imports torch / touches the GPU.
131         cue_flags: dict[str, bool] = Field(default_factory=dict)
132         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
133         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
134         min_crossings: int = Field(default=0, ge=0)
135         # Served Gate B: when the person cue is on, drop windows with median in-court
players
136         # < this. 0 = OFF.
137         min_players: int = Field(default=0, ge=0)
138         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
139         court_polygon: Optional[list[list[float]]] = None
140         # Net line for the person two-sided-motion split (person features only ? the shuttle
141         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
142         net_axis: Literal["x", "y"] = "y"
143         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
144
145
146     class IndexingConfig(BaseModel):
147         default_segmenter: str = "yolo_hybrid"
148         use_gpu: bool = True
149         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
150         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
151         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
152         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
153         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
154         detector_impl: str = "auto"
155         motion_threshold: float = 0.08
156         min_segment_duration: float = 3.0
157         dead_time_merge_gap: float = 2.0
158         chunk_duration: float = 90.0
159         chunk_overlap: float = 10.0
160         detector_model: str = "fasterrcnn_resnet50_fpn"
161         detector_score_threshold: float = 0.5
162         yolo_velocity_threshold: float = 0.15
163         yolo_frame_skip: int = 3
164         yolo_tracker: str = "bytetrack.yaml"
165         yolo_prefetch_workers: int = 2
166         min_action_window_duration: float = 2.0
167         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
168         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
169         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
170         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
171         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
172         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
173         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
174         max_window_duration: float = 6.0
175         window_min_split_gap: float = (
176             0.5 # min internal gap (s) that qualifies as a split point
177         )
178         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
179         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
180         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.

```



```

181      # Only cuts (recall can't drop). OFF by default until measured/promoted.
182      window_hard_cap: bool = False
183      # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
184      # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
185      # [?end] gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
186      # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
187      # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
188      # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, [?end] 4.96?3.31s (n=13).
The W1 cap
189      # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
190      window_inactivity_end: float = 1.5
191      inactivity_rally_thresh: float = 0.20
192      inactivity_min_density: float = 0.30
193      # R4 density DENOMINATOR fix (code-review finding B). The R4 trim's fast-fraction
denominator is
194      # the COUNT of active samples in the trailing window, which sparse tracking inflates
(one fast
195      # sample after a gap ? 100% dense ? tail held open). When True, the denominator is
the window's
196      # TEMPORAL length instead. No-op under dense tracking (sample-count == temporal-
span), so it only
197      # corrects the sparse-track case. *** P2b PROMOTED DEFAULT-ON (2026-06-29): golden-
set A/B at the
198      # SERVED operating point (n=15, 595 rallies) ? tolF1 0.353?0.364 (?tolF1 +0.010,
sign 10W/4L, NO
199      # regression), [?end] 2.97?2.72s, seg_count_ratio ?0.103 (p=0.006), split_rate
?0.010 (p=0.016),
200      # R1 over-seg guard PASS; served_regresses(eps=0.0)=False ?
promote_if_served_safe(structural)=True.
201      # See docs/RALLY_DETECTION_FIXES_PLAN.md §7/§8. ***
202      inactivity_temporal_density: bool = True
203      # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
204      # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
205      # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
206      # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
207      # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
208      # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
209      # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
210      # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
211      window_blob_fallback: bool = False
212      # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
213      # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
214      # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
215      # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
216      window_blob_factor: float = 4.0
217      log_yolo_candidates: bool = True
218      store_yolo_candidates: bool = True
219      ai_handoff_padding: float = 2.0

```

```

220     ai_max_workers: int = 5
221     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
222     # chunks of a high-bitrate (4K) source blow past the provider upload cap
223     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
224     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
225     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
226     # Matches gemini_refine's --clip-scale-width default.
227     ai_chunk_scale_width: int = 1280
228     # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
229     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
230     # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
231     # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
232     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
233     skip_ai_handoff: bool = False
234     # Optional debug knob: trim every video to the first N seconds before processing.
235     debug_duration: Optional[float] = None
236
237     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
238     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
239     fusion: FusionConfig = Field(default_factory=FusionConfig)
240
241     @field_validator("default_segmenter")
242     @classmethod
243     def segmenter_must_be_registered(cls, v: str) -> str:
244         # Lazy import: the segmenter registry imports modules that may import
245         # config, so we resolve it at validation time rather than at module load.
246         from backend.pipeline.segmenters.base import segmenter_registry
247
248         if segmenter_registry.get(v) is None:
249             available = sorted(segmenter_registry.list_available().keys())
250             raise ValueError(
251                 f"Segmenter '{v}' is not registered. Available: {' ', ' '.join(available)}"
252             )
253         return v
254
255     @model_validator(mode="after")
256     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
257         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
258         # step makes `while t < duration: t += step` loop forever, growing memory before
any
259         # GPU work. Reject the bad config at load time instead.
260         if self.chunk_overlap >= self.chunk_duration:
261             raise ValueError(
262                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
263                 f"({self.chunk_duration}); otherwise chunking never advances."
264             )
265         return self
266
267
268     class OutputConfig(BaseModel):
269         default_highlights_name: str = "highlights.mp4"
270         db_name: str = "sports_indexer.db"
271         # Directory where the DB, highlight reels, and processed clips are written.
272         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
273         output_dir: str = "output"
274
275
276     class WatermarkConfig(BaseModel):
277         """Branding overlay burned into each highlight clip during the per-clip re-encode

```

```

278 (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
279 (under `stitching.watermark` in config.json) to turn it off. The text is fully
280 configurable. The concat stage stays stream-copy (-c copy)."""
281
282 enabled: bool = True
283 text: str = "Generated by Khelsutra.guru"
284 logo_path: str = "" # optional transparent PNG overlay; "" = no logo
285 font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family below
286 font: str = "Sans" # fontconfig family used when font_path is empty
287 font_size_ratio: float = Field(
288     default=0.035, gt=0.0, le=0.5
289 ) # text height as a fraction of frame height
290 opacity: float = Field(default=0.85, ge=0.0, le=1.0)
291 box: bool = True # faint backing box behind the text for legibility
292 margin: int = Field(default=24, ge=0) # px inset from the chosen corner
293 position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
294     "bottom-right"
295 )
296
297
298 class StitchingConfig(BaseModel):
299     begin_padding: float = 0.5
300     end_padding: float = 0.5
301     use_cache: bool = False
302     min_shot_count: int = 1
303     # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
304     # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
305     # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
306     # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
307     # local detector over-fires and its false positives are mostly SHORT (motion blips,
308     # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
309     # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
310     min_rally_duration: float = Field(default=0.0, ge=0.0)
311     watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
312
313
314 class LoggingConfig(BaseModel):
315     """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
316
317     # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
318     # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
319     # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
320     # them to WARNING; set true to restore full chatter for request-flow debugging.
321     verbose_http: bool = False
322
323
324 class SecurityConfig(BaseModel):
325     # When set, /api/process video_path must resolve under this directory (hosted/tenant
326     # mode). Default None preserves the single-user local 'any local file' workflow.
327     allowed_video_root: Optional[str] = None
328
329     # --- Chunked upload (B2, PR-2) ---
330     # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
331     # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
332     # contain upload_dir or /api/process will reject the uploaded paths.
333     upload_dir: str = "output/uploads"
334     # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
335     # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
336     max_upload_mb: int = 4096
337     max_part_mb: int = 95

```

```

338     allowed_upload_extensions: List[str] = ["mp4", "mov"]
339
340     # --- M9 edge auth (Cloudflare Access, D9) ---
341     # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
342     # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
343     # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
344     # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
345     # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
346     # application's audience tag. NO secrets here ? these are public identifiers.
347     edge_auth_enabled: bool = False
348     cf_team_domain: str = (
349         "" # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
350     )
351     cf_access_aud: str = "" # the CF Access application AUD tag the token must carry
352     # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
353     # backend/main.py so importing the app does no filesystem I/O; default ["*"].
Credentials stay
354     # OFF ? Cf-Access is a header, not a cookie.)
355
356
357     class StorageConfig(BaseModel):
358         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
359         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
360         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
361         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
362         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
363         (boto3/google auth chains supply credentials). See ``backend/storage/``. """
364
365         backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
366         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
367         output_root: str = ""
368         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
369         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
370         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
371         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
372         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
373         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
374         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
375         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
376         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
377         input_root: str = ""
378         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
379         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
380         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
381         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).

```

```

When OFF
382     # the legacy flat layout is used unchanged.
383     single_folder_per_video: bool = False
384     # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
385     workspace_root: str = ""
386     # Cloud namespace prefix under the root so per-video folders sit tidily beside
387     # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
388     workspace_prefix: str = "workspaces"
389     local: dict[str, Any] = Field(default_factory=dict)
390     s3: dict[str, Any] = Field(default_factory=dict)
391     gcs: dict[str, Any] = Field(default_factory=dict)
392     gdrive: dict[str, Any] = Field(default_factory=dict)
393     # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
394     # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
395     gdrive_api: dict[str, Any] = Field(default_factory=dict)
396
397
398     class DeploymentConfig(BaseModel):
399         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
400         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
$4.3).
401
402         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
403         behaves exactly as before this section existed. A profile is opt-in (``config.json``
404         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
405         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
406         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
407         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
408
409         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
410
411         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
412         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
413         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
414         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
415         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
416
417
418     class BillingConfig(BaseModel):
419         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
420
421         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
422         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
423         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
424         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
425         inserts a real, calibrated version and points ``pricebook_version`` at it.
426
427         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
428         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
429         pricebook + the recording seam; usage emission (the metering hooks) is a separate

```

```

step."""
430
431     enabled: bool = False
432     pricebook_version: str = (
433         "placeholder-v0" # the seeded $0 version; owner flips to a real one
434     )
435     fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore market)
436     margin: float = 0.0 # resale markup (?0); 0 = bill at cost
437
438
439     class FlywheelConfig(BaseModel):
440         """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
441         loop: on a CONSENTED adult job, durably **capture** the source + rally output into
the per-video
442         workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden
TRAINING set, so
443         the owned model climbs toward the D11 ship floor.
444
445         **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-
identical to
446         today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
447         captures **nothing**, because ``flywheel.consent_attested`` currently returns
``False`` for every
448         job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-
delete-raw).
449         This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
450         wiring) + the owned-model + golden-label data needed to *activate* shadow-
eval/promotion are
451         owner/counsel/GPU scope. Pairs with M9-consent."""
452
453     enabled: bool = False
454     workspace_prefix: str = (
455         "flywheel" # per-video workspace prefix for captured artifacts
456     )
457     shadow_eval: bool = True # run the owned-vs-Gemini shadow comparison on capture
458     min_promote_n: int = 5 # promote_if_served_safe `min_n` (the served-safe gate)
459
460
461     class AppConfig(BaseModel):
462         """Root configuration object.
463
464         All runtime configuration should be accessed via an instance of this class
465         (or the loader) rather than raw dict lookups.
466         """
467
468         sport: Sport = "badminton"
469         ai_provider: str = "gemini"
470         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
471         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
472         ai_model: str = "gemini-flash-latest"
473
474         validation: ValidationConfig = Field(default_factory=ValidationConfig)
475         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
476         output: OutputConfig = Field(default_factory=OutputConfig)
477         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
478         security: SecurityConfig = Field(default_factory=SecurityConfig)
479         logging: LoggingConfig = Field(default_factory=LoggingConfig)
480         storage: StorageConfig = Field(default_factory=StorageConfig)
481         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
482         billing: BillingConfig = Field(default_factory=BillingConfig)

```

```

483         flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)
484
485         # Reject unknown keys ? typed config migration is complete (D3).
486         model_config = {
487             "extra": "forbid",
488             "validate_assignment": True,
489         }
490
491
492     # Convenience type alias for callers that want to annotate
493     Config = AppConfig
494

```

9. assistant (2026-06-29T17:01:58.287Z)

Now I have a clear picture. Let me confirm the production path: in `process\_video`, `idx\_cfg = self.config.indexing` where `self.config` is an `AppConfig` (typed), so `idx\_cfg` is a typed `IndexingConfig`, and `idx\_cfg.fusion` is a typed `FusionConfig`. Let me verify how `self.config` is built and confirm `BasePlaySegmenter.\_\_init\_\_` keeps it typed.

10. user (2026-06-29T17:01:59.674Z)

```

1     from abc import ABC, abstractmethod
2     from typing import Any, Dict, List, Optional, Type
3
4     from backend.infrastructure.database import Database
5     from backend.results import (
6         Result, # new standardized contract (Result[Value] == Success[Value] | Failure)
7     )
8
9
10    def _normalize_config(config: Any) -> Any:
11        """Normalize a plain-dict config into an AppConfig so segmenters can use typed
12        attribute access consistently. Production always passes AppConfig; tests may
13        pass plain dicts ? this bridge keeps both paths working."""
14        if isinstance(config, dict):
15            from backend.config.models import AppConfig
16
17            try:
18                return AppConfig(**config)
19            except Exception:
20                return (
21                    config # invalid config ? let the segmenter's own validation reject it
22                )
23        return config
24
25
26    class BasePlaySegmenter(ABC):
27        def __init__(self, db: Database, config: Dict[str, Any]):
28            self.db = db
29            self.config = _normalize_config(config)
30
31        @property
32        @abstractmethod
33        def name(self) -> str:
34            """Unique identifier for this segmenter."""
35            pass
36
37        @property
38        @abstractmethod
39        def description(self) -> str:
40            """Human-readable explanation of this segmenter."""
41            pass
42
43        @abstractmethod

```

```

44     def process_video(
45         self,
46         video_id: str,
47         video_path: str,
48         player_pool: List[str] = None,
49         max_frames: int = None,
50     ) -> "Result[List[Dict[str, Any]]]":
51         """
52         Processes a video to detect play segments, populates SQLite database,
53         and returns a Result.
54
55         Use isinstance(result, Failure) (or result.is_success()) to distinguish
56         "no segments found" from "something went wrong" (missing key, healthcheck
57         failure, etc.). This is the start of the standardized error/result contract.
58         """
59         pass
60
61
62     class SegmenterRegistry:
63     def __init__(self):
64         self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
65
66     def register(self, segmenter_cls: Type[BasePlaySegmenter]):
67         """Registers a new segmenter class under its name property."""
68         try:
69             temp_inst = segmenter_cls(None, {})
70             name = temp_inst.name
71         except Exception as e:
72             raise ValueError(
73                 f"Failed to resolve name property from segmenter class
74 {segmenter_cls.__name__}: {e}"
75             ) from e
76         self._segmenters[name] = segmenter_cls
77
78     def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
79         return self._segmenters.get(name)
80
81     def list_available(self) -> Dict[str, str]:
82         """Returns a dict of registered segmenter names and their descriptions."""
83         res = {}
84         for name, cls in self._segmenters.items():
85             try:
86                 temp_inst = cls(None, {})
87                 desc = temp_inst.description
88             except Exception:
89                 desc = "No description available."
90             res[name] = desc
91         return res
92
93     segmenter_registry = SegmenterRegistry()
94

```

11. user (2026-06-29T17:02:00.962Z)

```

backend/eval/served_gate_a.py:83:     seg = FusionSegmenter(None, {"indexing": idx_cfg, "sport":
"badminton"})
backend/pipeline/segmenters/fusion_hybrid.py:41:class
FusionSegmenter(TrajectoryHybridSegmenter):
---- how config is constructed in process_video callers ----
backend/pipeline/segmenters/base.py:29:         self.config = _normalize_config(config)

```

12. assistant (2026-06-29T17:02:07.022Z)

Now I understand the full picture. Let me trace the root cause precisely.

The key insight: `\_normalize\_config` calls `AppConfig(\*\*config)`. The config passed by `served\_gate\_a.py` is `{"indexing": idx\_cfg, "sport": "badminton"}` where `idx\_cfg` is a `**plain dict**`. So `AppConfig(\*\*{"indexing": <dict>, ...})` would normally coerce that nested dict into a typed `IndexingConfig`... unless it fails. Let me verify whether `AppConfig(\*\*config)` succeeds with this dict, because if it succeeds, `self.config.indexing` would be typed and `process\_video` would work ? but `served\_gate\_a` doesn't go through `process\_video`; it passes `idx\_cfg` (the raw dict) DIRECTLY to `\_enrich\_candidate\_stats`.

Let me confirm this is the actual bug by reproducing it.

13. user (2026-06-29T17:02:13.171Z)

```
self.config.indexing type: IndexingConfig
self.config.indexing.fusion type: FusionConfig
raw idx_cfg type passed to seams: dict
REPRO AttributeError: 'dict' object has no attribute 'fusion'
```

14. assistant (2026-06-29T17:02:20.380Z)

Reproduced exactly. The root cause is confirmed:

```
- **Production** (`process_video`): `idx_cfg = self.config.indexing` is a typed `IndexingConfig`
(because `_normalize_config` ran `AppConfig(**config)`, coercing the nested dict to typed). So
`idx_cfg.fusion` works, and `_enrich_candidate_stats` at line 135 (`fcfg = idx_cfg.fusion`)
succeeds.
- **`served_gate_a.served_handoff_windows`**: builds a plain `dict` `idx_cfg` and passes that
**raw dict** directly into `seg._enrich_candidate_stats(..., idx_cfg=idx_cfg_typed)` and
`seg._select_for_handoff(..., idx_cfg_typed)` ? bypassing `process_video` and
`_normalize_config`. The `idx_cfg_typed: Any = idx_cfg` line is just a type-checker silencer;
it's still a dict at runtime. Hence `dict.fusion` ? AttributeError at line 135.
```

Note `\_build\_runner` and `\_build\_native\_runner` already defensively coerce (`if isinstance(idx\_cfg, dict): idx\_cfg = IndexingConfig(\*\*idx\_cfg)`), but `\_enrich\_candidate\_stats` and `\_select\_for\_handoff` do NOT ? and `served\_gate\_a` only calls those two.

Let me verify the golden manifest exists and check the manifest the issue references, plus confirm what fields it carries.

15. user (2026-06-29T17:02:24.999Z)

```
-rw-r--r-- 1 avidullu avidullu 5456 Jun 29 11:00 output/p2b_golden/manifest_local.json
---- first spec ----
num specs: 15
keys of spec0: ['fps', 'frame_width', 'labels', 'name', 'trajectory']
present (traj+labels on disk): 15
names: ['adarsh_avi_singles', 'kushagra_singles', 'largetest_doubles', 'gbaaddy',
'testlarge_short', 'mahadevpura_singles', 'mahadevpura_2', 'GX010128', 'mahadevpura_1',
'GX030094', 'Badminton_BXH_2', 'GX010141', 'GX010137', 'Boxhill_Doubles', 'GX020094']
```

16. user (2026-06-29T17:02:28.837Z)

```
cv2 OK 4.13.0
---- does the default manifest exist? ----
ls: cannot access 'output/human_lovo_manifest.json': No such file or directory
```

17. assistant (2026-06-29T17:02:34.158Z)

This confirms everything:

```
- The golden manifest at `output/p2b_golden/manifest_local.json` exists with **15 specs, all 15
present** on disk (traj + labels), matching the test's "present 15-clip multicourt golden set"
and the `>= 6` gate.
- `cv2` is importable (4.13.0), so `pytest.importorskip("cv2")` passes on this box.
- The default manifest `output/human_lovo_manifest.json` does NOT exist, so without the
`SERVED_GATE_A_MANIFEST` env override the test still skips on this box.
```

Now let me verify the fix actually resolves the crash end-to-end by simulating the served path with the typed-coercion fix applied to the seam, and confirm which option is cleanest. Let me check the current behavior runs once the dict is coerced.

18. user (2026-06-29T17:02:41.246Z)

```
typed.fusion.min_crossings = 2
typed.fusion.cue_flags = {}
typed.wasb.velocity_threshold = 0.02
typed.dead_time_merge_gap = 2.0
typed.min_action_window_duration = 2.0
OK: extra=forbid accepted the dict (no unknown keys)
```

19. assistant (2026-06-29T17:02:46.746Z)

Option A (build a typed `IndexingConfig` in `\_served\_idx\_cfg`) works cleanly: the served dict's keys are all valid `IndexingConfig`/`FusionConfig`/`WasbIndexConfig` fields, so `extra="forbid"` accepts it. This is the preferred fix because it exercises the REAL production contract ? the production seams receive a typed `IndexingConfig` exactly as `process\_video` hands them (`idx\_cfg = self.config.indexing`).

Let me now do a final end-to-end smoke run of the actual served path on the golden manifest with a local monkeypatch of `\_served\_idx\_cfg` to return typed, confirming it produces the windows (and so I can report whether bands will move). I'll run a single video to keep it fast.

20. user (2026-06-29T17:03:00.284Z)

```
num_videos      = 15
mean_delta_f1   = -0.0233
mean_f1_off     = 0.2366
mean_f1_on      = 0.2134
mean_seg_off    = 0.5825
mean_seg_on     = 0.4696
f1_on/f1_off    = 0.9017
handoff_off>=on all: True
num_gt>0 all: True
```

21. assistant (2026-06-29T17:03:06.222Z)

Excellent. This is the decisive result. With Option A applied, the served litmus runs end-to-end on all 15 golden clips and produces:

```
- `mean_delta_f1 = -0.0233` ? the pinned band `-0.06 <= ... <= 0.03` **PASSES** (the test comment even cites "? -0.023").
- `mean_f1_on/mean_f1_off = 0.9017` ? `>= 0.5 * mean_f1_off` **PASSES** (test comment says "measured ratio ? 0.90").
- `mean_seg_ratio_on (0.4696) <= mean_seg_ratio_off (0.5825)` **PASSES**.
- `n_handoff_off >= n_handoff_on` for all, `num_gt > 0` for all ? **PASS**.
```

The served-path bands do NOT need re-pinning ? the 2026-06-22 snapshot reproduces exactly. Now let me verify the PROPOSAL-windowing test (the only one that doesn't touch the broken seam) still matches its pinned bands, since the issue asks me to note every pinned band.

22. user (2026-06-29T17:03:19.992Z)

```
PROPOSAL mean_delta = 0.0287 (pin: approx 0.03 abs 0.03 -> band [0.00,0.06])
PROPOSAL mean_seg_off = 1.3640 (pin: approx 1.36 abs 0.10 -> band [1.26,1.46])
PROPOSAL mean_seg_on  = 0.7919 (pin: approx 0.79 abs 0.10 -> band [0.69,0.89])
seg_on<seg_off: True
```

23. assistant (2026-06-29T17:03:25.413Z)

All PROPOSAL bands pass too (delta 0.0287 ? [0.00,0.06]; seg_off 1.364 ? [1.26,1.46]; seg_on 0.792 ? [0.69,0.89]). The PROPOSAL test never touches the broken seam (it calls `rally\_gate` directly), so it's already runnable ? but it shares the same `\_skip`/`\_SPECS` gate so it only

runs once a manifest is found.

I now have everything needed. Let me write the structured fix plan. The exact change for Option A:

24. user (2026-06-29T17:03:27.046Z)

```
29
30     from backend.eval import segmentation_metrics as _segm
31     from backend.eval.calibrate_wasb import load_golden_gts
32     from backend.eval.metrics import Interval, Score, score
33     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
```

25. assistant (2026-06-29T17:03:28.723Z)

I have everything I need. Let me produce the structured fix plan.

26. user (2026-06-29T17:04:19.168Z)

Structured output provided successfully